

# Reliable UDP Communication in Mininet

EEB24023 — Jitupan Mondal

## 1. Introduction

This assignment was based on building a simple reliable communication system over UDP inside Mininet. Since UDP does not guarantee delivery, reliability was added manually using sequence numbers, acknowledgements, timeout, retransmission, and duplicate detection. The final system used a Stop-and-Wait style approach between a client on `h2` and a server on `h1`.

## 2. Objective

The main objective was to implement a UDP client-server application in Mininet and add basic reliability features. The required mechanisms were:

- sequence numbers,
- ACK messages,
- timeout handling,
- retransmission of lost packets,
- duplicate detection at the server.

## 3. Network Setup

The default Mininet topology used in this assignment was:

$$h1 \text{ --- } s1 \text{ --- } h2$$

Here, `h1` acted as the server and `h2` acted as the client. The network was verified using `nodes`, `net`, and `pingall`. The connectivity check showed successful communication between the two hosts.

## 4. Protocol Working

The client sends one message at a time in the format:

SEQ|MESSAGE

For example:

3|Message 3 from h2

The server replies using an ACK in the format:

ACK|SEQ

For example:

ACK|3

The client waits for the ACK before sending the next message. If the ACK is not received within the timeout period, the same message is retransmitted. This forms a simple Stop-and-Wait reliability mechanism.

## 5. Implementation Summary

The program was implemented using Python UDP sockets. The server listens on the IP address of `h1`, receives packets, sends ACKs, and keeps track of duplicate messages using sequence numbers. The client sends 10 messages one by one, waits for the correct ACK, and retransmits whenever a timeout occurs.

### 5.1. Client Side

The client performs the following tasks:

- sends 10 messages,
- waits for ACK for each message,
- retransmits when timeout occurs,
- counts total packets sent,
- counts retransmissions,
- measures total transfer time.

### 5.2. Server Side

The server performs the following tasks:

- receives packets from the client,
- sends ACKs for each received sequence number,
- detects duplicate packets,
- counts unique messages,
- counts duplicate packets.

## 6. Timeout Rule

The timeout value was selected according to the last digit of the roll number. For roll number EEB24023, the last digit is 3, so the timeout used was:

$$0.7seconds$$

This timeout was used in all experiments.

## 7. Result Table

Three experiments were performed under different packet loss values in Mininet. The following table contains the final results.

Table 1: Experimental Results

| Roll No. | Name           | Loss (%) | Timeout (s) | Total Messages | Packets Sent | Retransmissions |
|----------|----------------|----------|-------------|----------------|--------------|-----------------|
| EEB24023 | Jitupan Mondal | 0        | 0.7         | 10             | 11           | 1               |
| EEB24023 | Jitupan Mondal | 5        | 0.7         | 10             | 10           | 0               |
| EEB24023 | Jitupan Mondal | 10       | 0.7         | 10             | 22           | 12              |

## 8. Screenshots

The following screenshots were captured and included in the submission folder:

- `nodes.png`
- `net.png`
- `pingall.png`
- `server_output.png`
- `client_output.png`

## 9. Discussion

The results show that packet loss directly affects reliability overhead. When loss is low, the client completes the transfer with very few retransmissions. As packet loss increases, more packets or ACKs are lost, which forces the client to retransmit and increases total transfer time. The server successfully detected duplicate packets using sequence numbers, which is important because retransmissions can make the same message arrive more than once.

## 10. Answers to Questions

### 10.1. Why is ACK needed?

ACK is needed to confirm that the receiver has successfully received a packet. Since UDP does not provide built-in reliability, the sender must rely on ACKs to know whether the data arrived correctly. If ACK is not received, the sender retransmits the packet.

### 10.2. Why is timeout needed?

Timeout is needed so that the sender does not wait forever for an ACK. If a packet or ACK is lost, timeout helps the sender detect the problem and retransmit the packet. This is essential for reliable delivery.

### 10.3. Why can duplicate messages occur?

Duplicate messages can occur when the sender does not receive the ACK in time and retransmits the same packet. In that case, the original packet may already have reached the server, but the ACK may have been lost. The server detects duplicates using the sequence number.

### 10.4. What happens when packet loss increases?

When packet loss increases, more packets or ACKs may be lost. This leads to more retransmissions, more duplicate packets, and a larger total transfer time. The overall efficiency of communication decreases.

### 10.5. How is this method similar to TCP?

This method is similar to TCP because both use sequence numbers, acknowledgements, and retransmissions to ensure reliability. Both protocols try to deliver data correctly even when the network is unreliable.

### **10.6. How is this method different from TCP?**

This method is much simpler than TCP. It only uses a Stop-and-Wait style mechanism, whereas TCP also includes connection establishment, flow control, congestion control, and efficient streaming of data. TCP is therefore more advanced and more efficient for general use.

## **11. Conclusion**

This assignment helped me understand how reliability can be built on top of UDP using sequence numbers, ACKs, timeout, and retransmission. The experiments showed that higher packet loss causes more retransmissions and longer transfer time. Overall, the project demonstrated the basic ideas behind reliable transport communication in a simple and practical way.